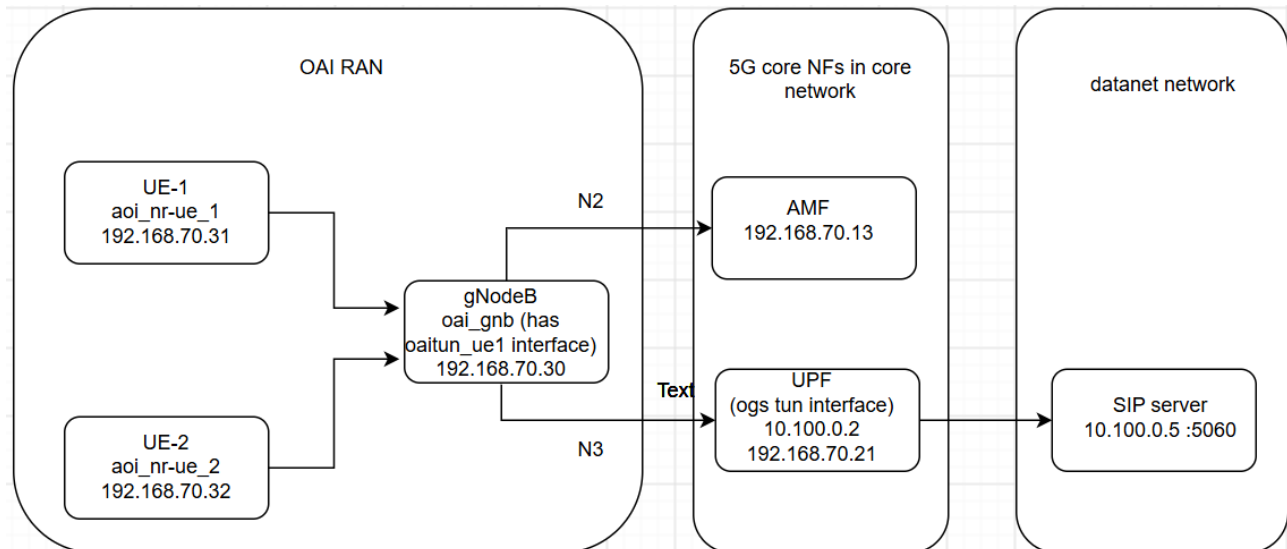


Open5GS

A complete 5G network simulation with an isolated testing space is set up using Docker Compose. It features two separate networks, a decoupled 5G core, and a dedicated data network acting as an external target to verify that internet traffic actually flows through the 5G data plane.



The Two Networks (The Bridges)

The architecture isolates signaling traffic from actual data traffic using two explicit Docker networks:

- **o5gs-core (192.168.70.0/24):** This is the internal network. Every 5G Core microservice (AMF, SMF, etc.), the database, the OAI gNodeB, and both virtual UEs sit on this subnet. This handles control signals and simulated over-the-air radio data via the loopback RF simulator.
- **o5gs-datanet (10.100.0.0/24):** This mimics the external public internet or an operator's Private Application Network. The sip-server sits here. Crucially, the UEs are not attached to this network. They can only reach it by routing packets through the 5G core network.

Component-by-Component Network Breakdown

1. The Database and Core Functions

- The Open5GS Network Functions (NFs) are cleanly broken down into separate microservice containers on the o5gs-core network:
- **mongo (192.168.70.10):** Holds user profiles.
- **Control Plane Components:** nrf (.11), amf (.13), ausf (.14), udm (.15), udr (.16), pcf (.17), bsf (.18), and nssf (.19). They communicate entirely within the 192.168.70.0/24 subnet.

2. The SMF and UPF (The Anchors)

- **smf (192.168.70.20)**: Manages subscriber sessions.
- **upf (192.168.70.21 AND 10.100.0.2)**: The User Plane Function acts as the bridge. It is the only container connected to **both** networks. Inside the UPF container, an entrypoint script spins up a virtual adapter called ogstun inside a private subnet (10.45.0.1/16).
- It configures IP masquerading (iptables -t nat -A POSTROUTING...). This translates IP traffic arriving from the UEs over the 5G network into its own address so it can route them cleanly out to the o5gs-datanet.

3. The OAI RAN Layer (gNodeB and UEs)

- **oai-gnb (192.168.70.30)**: Listens on the core network. It targets the AMF container for standard N2 control signaling. Because it has `--rfsim --rfsimulator.serveraddr server` turned on, it opens a local UDP socket on this IP to receive simulated radio frames.
- **oai-nr-ue-1 (192.168.70.31) & oai-nr-ue-2 (192.168.70.32)**: Both UEs live on the core network. The flag `--rfsimulator.serveraddr 192.168.70.30` tells them to send their virtual over-the-air radio blocks straight to the gNodeB's IP.

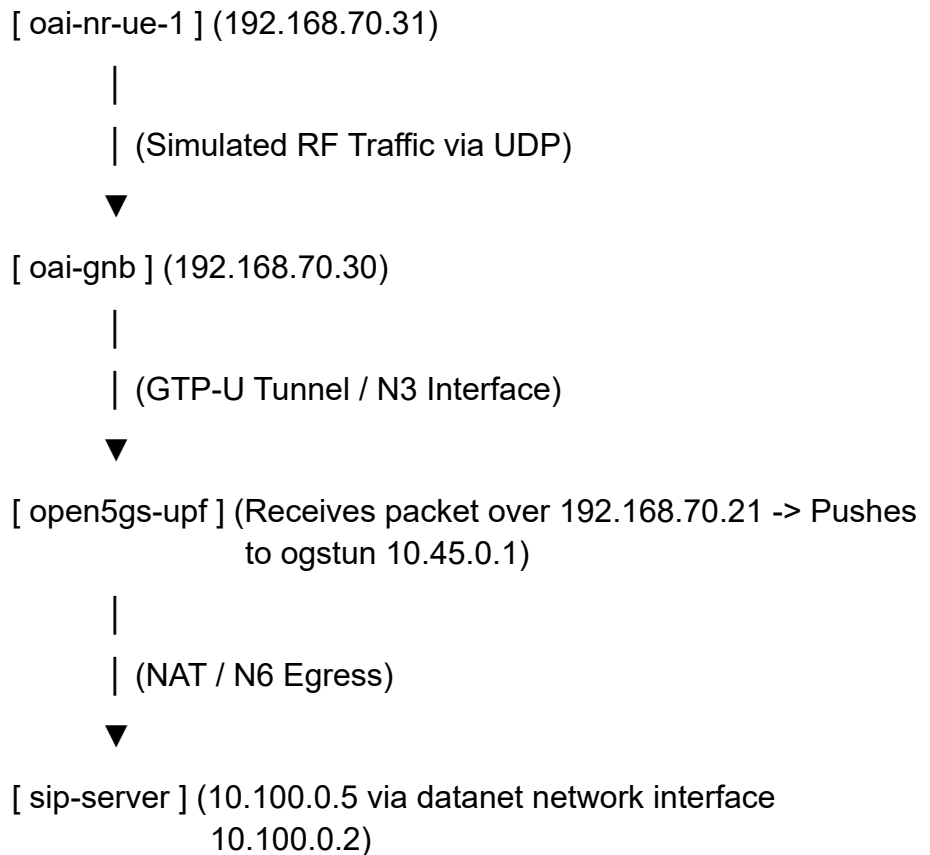
4. The Isolated Application Layer

- **sip-server (10.100.0.5)**: Lives entirely on the o5gs-datanet subnet. It has no physical link to the 192.168.70.0/24 network.

1) End-to-End Packet Path (How Traffic Moves)

Because the UEs are not directly connected to the o5gs-datanet, an application like sip-client-1 running out of a UE cannot use Docker's native bridge network to reach the SIP server. Instead, data must travel down the entire 5G network stack:

- The UE signs onto the network and is assigned an internal IP belonging to the 10.45.0.0/16 pool by the Open5GS SMF.
- A SIP packet targeting 10.100.0.5 leaves the UE client application.
- The UE encapsulates this packet inside an RF simulator wrapper and streams it over UDP to the gNodeB (192.168.70.30).
- The gNodeB strips away the RF simulator wrapper, packs the data into a GTP-U tunnel, and forwards it to the UPF (192.168.70.21).
- The UPF unpacks the GTP-U tunnel, handles Network Address Translation (NAT) via iptables, and pushes the packet out through its second network card directly onto the o5gs-datanet network to hit destination 10.100.0.5.



Prereq: docker-compose.yaml (docker compose up -d)

Script to verify: scripts/verify.sh

At least two UEs must successfully attach.

docker compose logs amf | grep -Ei "imsi-00101|Registration complete"

```

pragati@DESKTOP-74FRQK3:Open5GS$ docker compose logs amf | grep -Ei "imsi-00101|Registration complete"
amf | 07/24 10:44:03.766: [gmm] INFO: [imsi-001010000000002] Registration complete (./src/amf/gmm-sm.c:2658)
amf | 07/24 10:44:03.766: [amf] INFO: [imsi-001010000000002] Configuration update command (./src/amf/nas-path.c:607)
amf | 07/24 10:44:03.768: [gmm] INFO: UE SUPI[imsi-001010000000002] DNN[internet] S_NSSAI[SST:1 SD:0xffffffff] smContextR
ef[NULL] smContextResourceURI[NULL] (./src/amf/gmm-handler.c:1374)
amf | 07/24 10:44:03.811: [gmm] INFO: [imsi-001010000000001] Registration complete (./src/amf/gmm-sm.c:2658)
amf | 07/24 10:44:03.811: [amf] INFO: [imsi-001010000000001] Configuration update command (./src/amf/nas-path.c:607)
amf | 07/24 10:44:03.812: [gmm] INFO: UE SUPI[imsi-001010000000001] DNN[internet] S_NSSAI[SST:1 SD:0xffffffff] smContextR
ef[NULL] smContextResourceURI[NULL] (./src/amf/gmm-handler.c:1374)
amf | 07/24 10:44:03.831: [amf] INFO: [imsi-001010000000002:10:11][0:0:NULL] /nsmf-pdusession/v1/sm-contexts/{smContext
Ref}/modify (./src/amf/nsmf-handler.c:942)
amf | 07/24 10:44:03.856: [amf] INFO: [imsi-001010000000001:10:11][0:0:NULL] /nsmf-pdusession/v1/sm-contexts/{smContext
Ref}/modify (./src/amf/nsmf-handler.c:942)
pragati@DESKTOP-74FRQK3:Open5GS$

```

docker compose logs oai-nr-ue-1 | grep -Ei "Registration Accept|PDU Session Establishment Accept"

docker compose logs oai-nr-ue-2 | grep -Ei "Registration Accept|PDU Session Establishment Accept"

```

pragati@DESKTOP-74FRQK3:Open5GS$docker compose logs oai-nr-ue-1 | grep -Ei "Registration Accept|PDU Session Establishmen
t Accept"
oai-nr-ue-1 | 31003.250791 [NAS] I Received Registration Accept with result 3GPP
oai-nr-ue-1 | 31003.294513 [NAS] I Received PDU Session Establishment Accept, UE IPv4: 10.45.0.3
pragati@DESKTOP-74FRQK3:Open5GS$docker compose logs oai-nr-ue-2 | grep -Ei "Registration Accept|PDU Session Establishmen
t Accept"
oai-nr-ue-2 | 31003.194656 [NAS] I Received Registration Accept with result 3GPP
oai-nr-ue-2 | 31003.260438 [NAS] I Received PDU Session Establishment Accept, UE IPv4: 10.45.0.2
pragati@DESKTOP-74FRQK3:Open5GS$

```

The UE have working IP connectivity through the 5G core:

Each UE has an IP from the core's pool (10.45.0.0/16)

docker exec oai-nr-ue-1 ip addr show oaitun_ue1

docker exec oai-nr-ue-2 ip addr show oaitun_ue1

```

pragati@DESKTOP-74FRQK3:Open5GS$docker exec oai-nr-ue-1 ip addr show oaitun_ue1
3: oaitun_ue1: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.45.0.3/24 scope global oaitun_ue1
        valid_lft forever preferred_lft forever
    inet6 fe80::26cc:e22e:112a:fa0b/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
pragati@DESKTOP-74FRQK3:Open5GS$docker exec oai-nr-ue-2 ip addr show oaitun_ue1
3: oaitun_ue1: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.45.0.2/24 scope global oaitun_ue1
        valid_lft forever preferred_lft forever
    inet6 fe80::5188:eea2:deb3:9344/64 scope link stable-privacy
        valid_lft forever preferred_lft forever

```

Ping the UPF gateway THROUGH the tunnel (traffic crosses gNB -> N3 -> UPF)

docker exec oai-nr-ue-1 ping -c 4 -I oaitun_ue1 10.45.0.1

docker exec oai-nr-ue-2 ping -c 4 -I oaitun_ue1 10.45.0.1

```

pragati@DESKTOP-74FRQK3:Open5GS$docker exec oai-nr-ue-1 ping -c 4 -I oaitun_ue1 10.45.0.1
PING 10.45.0.1 (10.45.0.1) from 10.45.0.3 oaitun_ue1: 56(84) bytes of data.
64 bytes from 10.45.0.1: icmp_seq=1 ttl=64 time=71.5 ms
64 bytes from 10.45.0.1: icmp_seq=2 ttl=64 time=54.2 ms
64 bytes from 10.45.0.1: icmp_seq=3 ttl=64 time=64.8 ms
64 bytes from 10.45.0.1: icmp_seq=4 ttl=64 time=50.2 ms

--- 10.45.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2997ms
rtt min/avg/max/mdev = 50.164/60.165/71.503/8.461 ms
pragati@DESKTOP-74FRQK3:Open5GS$docker exec oai-nr-ue-2 ping -c 4 -I oaitun_ue1 10.45.0.1
PING 10.45.0.1 (10.45.0.1) from 10.45.0.2 oaitun_ue1: 56(84) bytes of data.
64 bytes from 10.45.0.1: icmp_seq=1 ttl=64 time=46.8 ms
64 bytes from 10.45.0.1: icmp_seq=2 ttl=64 time=45.5 ms
64 bytes from 10.45.0.1: icmp_seq=3 ttl=64 time=51.5 ms
64 bytes from 10.45.0.1: icmp_seq=4 ttl=64 time=44.2 ms

```

Reach the external data network (SIP server) through the core:

docker exec oai-nr-ue-1 sh -c 'ip route replace 10.100.0.0/24 dev oaitun_ue1; ping -c 4 10.100.0.5'

```
pragati@DESKTOP-74FRQK3:Open5GS$docker exec oai-nr-ue-1 sh -c 'ip route replace 10.100.0.0/24 dev oaitun_ue1; ping -c 4 10.100.0.5'
PING 10.100.0.5 (10.100.0.5) 56(84) bytes of data.
64 bytes from 10.100.0.5: icmp_seq=1 ttl=63 time=47.3 ms
64 bytes from 10.100.0.5: icmp_seq=2 ttl=63 time=40.1 ms
64 bytes from 10.100.0.5: icmp_seq=3 ttl=63 time=22.6 ms
64 bytes from 10.100.0.5: icmp_seq=4 ttl=63 time=27.2 ms
```

`docker compose logs sip-client-1 | grep -E "ping OK|SUCCESS: SIP"`

`docker compose logs sip-client-2 | grep -E "ping OK|SUCCESS: SIP"`

```
pragati@DESKTOP-74FRQK3:Open5GS$docker compose logs sip-client-1 | grep -E "ping OK|SUCCESS: SIP"
sip-client-1 | [sip-client] ping OK -> IP connectivity through the 5G core works
sip-client-1 | [sip-client] SUCCESS: SIP dialog(s) completed over the 5G data plane
pragati@DESKTOP-74FRQK3:Open5GS$docker compose logs sip-client-2 | grep -E "ping OK|SUCCESS: SIP"
sip-client-2 | [sip-client] ping OK -> IP connectivity through the 5G core works
sip-client-2 | [sip-client] SUCCESS: SIP dialog(s) completed over the 5G data plane
pragati@DESKTOP-74FRQK3:Open5GS$
```

2) SIP-Based Voice Services on Top of the 5G Data Plane

The Docker Compose configuration (**docker-compose.ott.yaml**) adds a SIP based voice service on top of the 5G data plane. It uses Kamailio to handle call routing and [RTPengine](#) to stream live audio through the 5G data path from the microphone to the speaker. Both services attach to a dedicated data network (datanet) with fixed local IP addresses.

- **Kamailio**: Acts as the call manager (SIP registrar and proxy). It handles call setup, ringing, and teardown instructions at IP 10.100.0.10.
- **RTPengine**: Acts as the media relay. It processes the actual audio stream packets (RTP) moving between callers at IP 10.100.0.11.
- **phone-ue1 / phone-ue2**: Mock user equipment nodes (virtual phones- Linphone) that simulate a live call over the 5G User Plane Function (UPF) network stack.

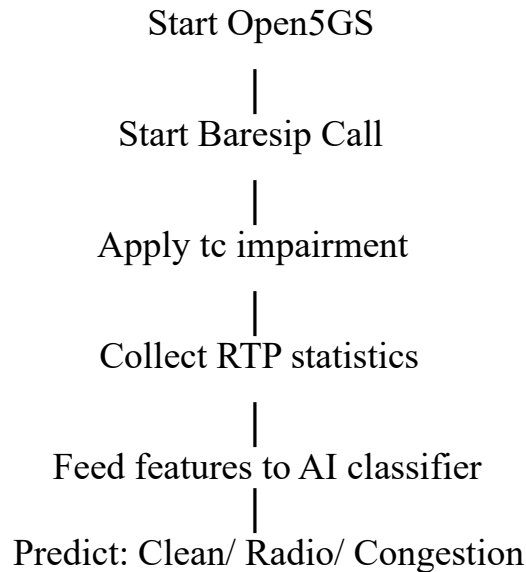
Logs saved at : **logs/kamailio.logs , logs/rtpengine.logs**

3) AI based RTP classifier:

The classifier analyzes RTP statistics such as **packet loss, jitter, and one-way delay**. Based on their patterns, it predicts the likely cause:

- **Clean**: Low loss, low jitter, stable delay → no action.
- **Radio impairment**: Packet loss is high but delay and jitter stay relatively stable → enable FEC/redundancy and rely on PLC.
- **Congestion**: Delay and jitter increase before packet loss appears → reduce bitrate or switch to a lower-bandwidth codec.

This approach makes the adaptation **cause-aware**: instead of treating all quality degradation the same way, the system selects the remedy that best matches the underlying network condition.



Network impairments were simulated on the oaitun_ue1 network interface using Linux Traffic Control (tc).

- 1) tc qdisc add dev oaitun_ue1 root netem loss 7% 30 (7% packet loss – Radio)
- 2) tc qdisc add dev oaitun_ue1 root tbf rate 56kbit burst 2kb latency 500ms (limit the bandwidth – congestion)

Feature extraction and model training:

Features are computed from the **RTP packets** in each **1-second analysis window**. The program (scripts/rtp-train.sh) first extracts information from every RTP packet and trains classifier model(logistic regression methodology):

- **Arrival time** (t) – when the packet was captured.
- **Sequence number** (seq) – used to detect missing or out-of-order packets.
- **RTP timestamp** (ts) – indicates when the audio sample was generated.
- **Packet size** (size) – used to estimate bitrate.

Using these values, it calculates the following features:

pps (Packets Per Second), kbps (Kilobits Per Second), loss_rate, reorder_rate, jitter_mean, jitter_max, ia_mean (Inter-arrival Mean), ia_std (Inter-arrival Standard Deviation), slope (owd_slope_ms_per_s), owd_range, burst_mean_len, burst_frac

ML steps:

Prereqs:

- base 5G up with both UEs attached, and kamailio + rtpengine up.

```
docker compose -f docker-compose.yaml -f docker-compose.sip.yaml down -v
docker compose -f docker-compose.yaml -f docker-compose.sip.yaml build
docker compose -f docker-compose.yaml -f docker-compose.sip.yaml up -d
```

- host has python3 + numpy: `python3 -c 'import numpy'`

a) Generate a LABELED dataset over 5G

-- bash scripts/rtp-dataset.sh

Drives baresip calls and impairs the caller's tunnel with tc, capturing the received stream at the callee (ue1). The call is established BEFORE the impairment is applied, so SIP setup is never affected.

Produces: ml/data/clean.pcap(no impairment)

ml/data/radio.pcap (tc netem loss 7% 30% = random link loss)

ml/data/congestion.pcap (tc tbf rate 56kbit = queue build-up)

b) Extract features + train + evaluate

-- bash scripts/rtp-train.sh

Extracts per-1s RTP features from every ml/data/*.pcap (label = filename before the first '-'), trains the numpy softmax classifier, and prints the class balance, held-out accuracy, confusion matrix and top features.

Outputs: ml/model.npz and ml/data/dataset.csv

c) Classify a capture and get the recommended mitigation

-- cd ml

--python3 predict.py data/congestion.pcap # -> verdict: congestion / REDUCE
bitrate

--python3 predict.py data/radio.pcap # -> verdict: radio / ADD FEC